Algorítmica

Tema 3. Algoritmos voraces (*greedy*)

Francisco Javier Cabrerizo Lorite

Curso académico 2024-2025

ETS de Ingenierías Informática y de Telecomunicación. Universidad de Granada

Índice

1. Introducción

2. Características generales

3. Ejemplos de aplicación

4. Teoría de grafos

5. Ejemplos de aplicación sobre grafos

6. Consideraciones finales

Objetivos

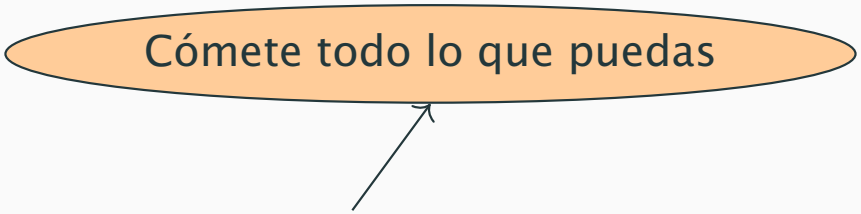
- Saber identificar soluciones distintas para un problema y evaluar su bondad.
- Ser consciente de la importancia del análisis de la eficiencia de un algoritmo como paso previo a su implementación.
- Entender la técnica voraz (*greedy*) de resolución de problemas y los distintos casos que pueden presentarse (solución óptima, no óptima, o no obtención de la solución).
- Conocer los criterios de aplicación de la técnica de diseño de algoritmos voraces (*greedy*).
- Conocer los algoritmos de Kruskal, Prim y Dijkstra.

Introducción

Introducción

Greedy

- Sinónimo de voraz, ávido, glotón, etc.





- Son algoritmos no previsores (ya se verá):
  - Pan para hoy y hambre para mañana.
  - Dios proveerá.
  - Reventar antes de que sobre.

Características

- Construyen la solución paso a paso (por etapas).
- En cada etapa, «toman el mejor bocado que pueden tragar», sin analizar las posibles consecuencias (voracidad).
- Las decisiones se toman únicamente en base a la información disponible en cada etapa (miopía).
- No vuelven a considerar las decisiones ya tomadas ni las modifican en posteriores etapas.
- Requieren una función objetivo que determine el valor de la solución obtenida.

Aplicación: problemas de optimización

- Dado un problema con n entradas, el objetivo es obtener un subconjunto de estas que satisfaga una determinada restricción definida en el problema.
- Cada uno de los subconjuntos que cumple las restricciones es una solución factible o prometedora.
- Una solución factible que maximiza o minimiza una determinada función objetivo se denomina solución óptima.
- Este tipo de problemas se suele resolver mediante una secuencia de pasos, donde en cada uno de ellos existe un conjunto de opciones posibles.
- Los algoritmos voraces resuelven estos problemas mediante la elección de óptimos locales con la esperanza de que estos conduzcan a un óptimo global.

Introducción

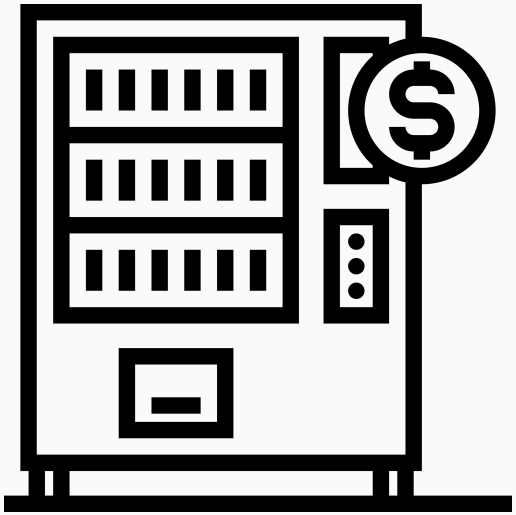
Ventajas

- Eficientes.
- Fáciles de diseñar.
- Fáciles de implementar.

Desventajas

- No garantizan que se obtenga siempre la solución óptima.
- Podrían no obtener la solución, aunque esta exista.
- Si no se puede demostrar que obtienen el óptimo, es un enfoque poco elegante.

«El cambio es inevitable, excepto para una máquina expendedora»



Problema del cambio de monedas

Una máquina expendedora debe devolver un cierto cambio *n* con el mínimo número de monedas. Se asume que la máquina tiene un número infinito de monedas de céntimos (1, 2, 5, 10, 20, 50) y de euros (1, 2).

¿Cómo se resolvería este problema?

Problema del cambio de monedas: algoritmo

```
función devolver_cambio(n)
    M ← {200, 100, 50, 20, 10, 5, 2, 1} // monedas de la máquina
    S ← ∅ // contendrá las monedas a devolver
    suma ← 0 // suma de los elementos de S
    mientras suma ≠ n hacer
        x ← mayor elemento de C tal que suma + x ≤ n
        si no existe tal elemento entonces
            devolver «Solución no encontrada»
        si no
            S ← S ∪ {moneda de valor x}
            suma ← suma + x
        fin si
    fin mientras
    devolver S
fin función
```

Introducción

¿El algoritmo para el problema del cambio de monedas es óptimo?

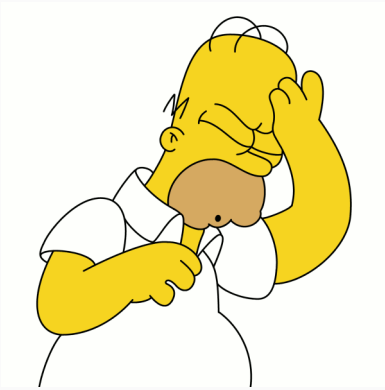
- Depende del sistema monetario.
- $M = \{100, 50, 25, 10, 1\}$ y $n = 40$.

¿Devuelve siempre solución?

- Depende del número de monedas que queden.
- 3 monedas de 50, 5 monedas de 20 y 12 monedas de 1.
- $n = 63$.

¡Cuidado!

- Dice que no hay solución.
- Sí la hay: 3 monedas de 20 y 3 monedas de 1.



Características generales

Diseño de un algoritmo voraz: elementos (1/2)

- **Conjunto (o lista) de candidatos.** Representa las posibles decisiones que se pueden tomar en cada momento para obtener la solución del problema.
- **Conjunto (o lista) de candidatos elegidos.** Representa las decisiones consideradas y elegidas como parte de la solución hasta ese momento.
- **Conjunto (o lista) de candidatos descartados.** Representa las decisiones consideradas y descartadas hasta ese momento.
- **Función solución.** Determina si las decisiones tomadas (candidatos elegidos) hasta ese momento proporcionan una solución al problema (no necesariamente óptima).

Diseño de un algoritmo voraz: elementos (2/2)

- **Función de factibilidad.** Comprueba si un cierto conjunto de candidatos es factible (es posible completar el conjunto añadiendo otros candidatos para obtener una solución, no necesariamente óptima). Se aplica a los candidatos que forman actualmente la solución junto con el candidato más prometedor.

- **Función de selección.** Determina en cada momento el candidato más prometedor para formar la solución de entre los que aún no han sido elegidos ni descartados.

- **Función objetivo.** Proporciona el valor de la solución obtenida (valor de lo que se quiere optimizar).

Problema del cambio de monedas: elementos

- **Conjunto (o lista) de candidatos:** conjunto de monedas, con tantas monedas de cada valor que nunca se agotan (sin embargo, debe ser finito).
- **Conjunto (o lista) de candidatos elegidos:** las monedas que se han ido eligiendo para ser devueltas.
- **Función solución:** comprueba si el valor de las monedas seleccionadas hasta el momento es exactamente el valor que hay que devolver.
- **Función de factibilidad:** comprueba si la suma del valor de un conjunto de monedas no sobrepasa la cantidad que hay que devolver.
- **Función de selección:** selecciona la moneda de valor mayor de las que quedan en el conjunto de candidatos.
- **Función objetivo:** cuenta el número de monedas utilizadas en la solución.

Diseño de un algoritmo voraz: funcionamiento

- El algoritmo voraz procede paso por paso. Inicialmente, el conjunto de candidatos elegidos (solución) está vacío.
- En cada paso, se considera añadir a este conjunto el mejor candidato, sin considerar los restantes (función de selección y función objetivo).
- Si el conjunto ampliado de candidatos elegidos no es factible, se rechaza el candidato devuelvo por la función de selección. Sin embargo, si el conjunto ampliado es factible, se añade al conjunto de candidatos elegidos (solución), en donde pasará a estar desde ahora hasta el final (función de factibilidad).
- Cada vez que se amplía el conjunto de candidatos elegidos, se comprueba si este constituye una solución al problema (función solución).
- Si el algoritmo voraz funciona «correctamente», la primera solución que encuentra es siempre óptima.

Diseño de un algoritmo voraz: esquema

```
función voraz(C)
    // C es el conjunto de candidatos
    S ← ∅ // se construye la solución en el conjunto S
    mientras C ≠ ∅  &  no solución(S) hacer
        x ← seleccionar(C)
        C ← C \ {x}
        si factible(S ∪ {x}) entonces
            S ← S ∪ {x}
        fin si
    fin mientras
    si solución(S) entonces
        devolver C
    si no
        devolver «No existen soluciones»
    fin si
fin función
```

Diseño de un algoritmo voraz: óptimo

- Si no encuentra la solución óptima, basta encontrar un «contraejemplo» para demostrar que no la proporciona.

- Si proporciona la solución óptima, se debe demostrar con alguna de las siguientes metodologías de demostración:

  - **Demostración por contradicción.** Se supone que una proposición es falsa. Se demuestra que esta hipótesis implica que una propiedad que se sabe que es cierta, es falsa (absurdo o contradicción). Se concluye que la proposición es verdadera.

  - **Demostración por inducción matemática.** Se demuestra que la proposición es cierta para un caso base (base de la inducción). Entonces, se supone que la hipótesis que se quiere demostrar es cierta hasta k. Si se demuestra que es cierta para $k + 1$ (paso inductivo), es cierta en todos los casos.

Ejemplos de aplicación

El problema de la mochila

Enunciado

Considérense n objetos con pesos $p_1, p_2, \ldots, p_n$ y beneficios $b_1, b_2, \ldots, b_n$ y una mochila capaz de albergar un peso máximo P. Se quiere llenar la mochila de forma que se maximice el beneficio de los objetos incluidos sin sobrepasar su peso máximo.

En esta versión del problema, se asume que los objetos se pueden dividir en trozos más pequeños, de forma que se puede elegir llevar una fracción x_i del objeto i, con $0 \leq x_i \leq 1$.

Formalmente, el problema se define como:

$$\text{maximizar} \quad \sum_{i=1}^{n} x_i b_i \qquad \text{con la restricción} \quad \sum_{i=1}^{n} x_i p_i \leq P$$

con $b_i > 0$, $p_i > 0$ y $0 \leq x_i \leq 1$ para $1 \leq i \leq n$.

El problema de la mochila

Consideraciones

- Si $\sum_{i=1}^{n} p_i \leq P$, lo óptimo consiste en introducir todos los objetos en la mochila.
- Se puede asumir por tanto que en las instancias interesantes del problema se tiene que $\sum_{i=1}^{n} p_i > P$.
- La solución óptima debe utilizar todo el peso de la mochila (siempre se puede introducir una fracción de uno de los objetos restantes e incrementar así el beneficio).
- Por tanto, una solución óptima cumple que $\sum_{i=1}^{n} x_i p_i = P$.
- La estrategia de un algoritmo voraz consistirá en seleccionar cada objeto por turno siguiendo algún orden adecuado, introducir la mayor fracción posible del objeto en la mochila, y parar cuando la mochila esté llena.

El problema de la mochila

Diseño del algoritmo voraz: elementos

- **Conjunto de candidatos:** los n objetos.
- **Conjuntos de candidatos elegidos:** la solución es un vector $(x_1, x_2, \cdots, x_n)$ que indica qué fracción de cada objeto se mete en la mochila.
- **Función solución:** comprueba que la mochila esté totalmente llena:

$$\sum_{i=1}^{n} x_i p_i = P$$

- **Función de factibilidad:** el peso de las fracciones de los objetos metidos en la mochila hasta el momento más el peso de la fracción del objeto más prometedor no supera el peso total de la mochila.
- **Función objetivo:** beneficio total de los objetos introducidos en la mochila:

$$\sum_{i=1}^{n} x_i b_i$$

El problema de la mochila

Diseño del algoritmo voraz: función de selección

- **Estrategia 1.** Seleccionar el objeto que proporciona mayor beneficio de los restantes (se incrementa el beneficio de la mochila del modo más rápido posible).

- **Estrategia 2.** Seleccionar el objeto de menor peso de los restantes (la capacidad se agota de la forma más lenta posible).

- **Estrategia 3.** Seleccionar el objeto cuyo beneficio por unidad de peso sea el mayor posible de los restantes.

Diseño de un algoritmo voraz: pseudocódigo

```
función mochila(p[1..n],b[1..n], P)
    // inicialización
    para todo i = 1, . . . , n hacer
        x[i] ← 0 // solución
    fin para todo
    peso_actual ← 0
    // bucle voraz
    mientras peso_actual < P hacer // función solución
        i ← el mejor objeto restante // función de selección
        si peso_actual + p[i] ≤ P entonces
            x[i] ← 1
            peso_actual ← peso_actual + p[i]
        si no
            x[i] ← (P − peso_actual) / p[i]
            peso_actual ← P
        fin si
    fin mientras
    devolver x
fin función
```

- $T(n) \in \mathrm{O}(n \log(n))$

Ejemplo de aplicación en una instancia concreta

- Sea $P = 100$ y $n = 5$. Además, los pesos y beneficios de los objetos son los siguientes:

	1	2	3	4	5
p	10	20	30	40	50
b	20	30	66	40	60
b/p	2.0	1.5	2.2	1.0	1.2

- Resultado según el criterio de selección empleado:

	x_1	x_2	x_3	x_4	x_5	**Beneficio**
Máximo b_i	0	0	1	0.5	1	**146**
Mínimo p_i	1	1	1	1	0	**156**
Máximo b_i/p_i	1	1	1	0	0.8	**164**

El problema de la mochila

Teorema

Si se seleccionan los objetos por orden decreciente de b_i/p_i, el algoritmo de la mochila encuentra una solución óptima.

Demostración (1/3)

- Supongamos, sin pérdida de generalidad, que los objetos disponibles están ordenados de la siguiente forma:

$$b_1/p_1 \geq b_2/p_2 \geq \ldots \geq b_n/p_n$$

- Sea $X = (x_1, x_2, \ldots, x_n)$ la solución obtenida por el algoritmo voraz:
  - Si todos los x_i son iguales a 1, la solución es óptima.
  - En otro caso, sea j el menor índice tal que $x_j < 1$. Por la forma en que trabaja el algoritmo, $x_i = 1$ cuando $i < j$, $x_i = 0$ cuando $i > j$, y $\sum_{i=1}^{n} x_i p_i = P$.

- Sea $B(X) = \sum_{i=1}^{n} x_i b_i$ el beneficio que se obtiene para la solución X.

Demostración (2/3)

- Ahora, sea $Y = (y_1, y_2, \ldots, y_n)$ cualquier solución factible, y sea su beneficio $B(Y) = \sum_{i=1}^{n} y_i b_i$.

- Por ser solución factible, cumple que $\sum_{i=1}^{n} y_i p_i \leq P$. Entonces, puesto que $\sum_{i=1}^{n} x_i p_i = P$, restando ambas capacidades, podemos afirmar que:

$$\sum_{i=1}^{n} (x_i p_i - y_i p_i) = \sum_{i=1}^{n} (x_i - y_i) p_i \geq 0$$

- La diferencia de beneficios es:

$$B(X) - B(Y) = \sum_{i=1}^{n} (x_i - y_i) b_i = \sum_{i=1}^{n} (x_i - y_i) p_i \frac{b_i}{p_i}$$

Demostración (3/3)

- Considérese ahora de nuevo el menor índice j tal que $x_j < 1$ en la solución X obtenida mediante el algoritmo voraz:
  - Si $i < j$, entonces $x_i = 1$ y, por tanto, $(x_i - y_i) \geq 0$. Además, por la forma en que se han ordenado los objetos, $\frac{b_i}{p_i} \geq \frac{b_j}{p_j}$.
  - Si $i > j$, entonces $x_i = 0$ y, por tanto, $(x_i - y_i) \leq 0$. Además, por la forma en que se han ordenado los objetos, $\frac{b_i}{p_i} \leq \frac{b_j}{p_j}$.
  - Si $i = j$, entonces $\frac{b_i}{p_i} = \frac{b_j}{p_j}$.
- Así pues, en todos los casos se tiene que $(x_i - y_i)\frac{b_i}{p_i} \geq (x_i - y_i)\frac{b_j}{p_j}$. Por tanto:

$$B(X) - B(Y) = \sum_{i=1}^{n} (x_i - y_i) p_i \frac{b_i}{p_i} \geq \frac{b_j}{p_j} \sum_{i=1}^{n} (x_i - y_i) p_i \geq 0$$

es decir, $B(X) \geq B(Y)$, tal como se quería demostrar.

El problema de la mochila

Versión alternativa: el problema de la mochila 0-1

- El conjunto solución no puede contener fracciones de objetos, es decir, se tiene que $x_i \in \{0, 1\}$ para todo $i = 1, 2, \ldots, n$.

- Como en la versión anterior, el objetivo es:

$$\text{maximizar} \quad \sum_{i=1}^{n} x_i b_i \qquad \text{con la restricción a} \quad \sum_{i=1}^{n} x_i p_i \leq P$$

  con $b_i > 0$ y $p_i > 0$.

¿Seguirá en esta versión dando la solución óptima el algoritmo voraz propuesto anteriormente?

Versión alternativa: el problema de la mochila 0-1

- El algoritmo voraz no obtiene el óptimo en esta versión del problema.
- **Contraejemplo.** Supongamos una mochila con capacidad $P = 6$ y 3 objetos con los siguientes pesos y beneficios:

	1	2	3
p	5	3	3
b	11	6	6
b/p	2.2	2	2

- El algoritmo voraz elige como mejor candidato el primer objeto, y como ya no caben más, la solución tiene un beneficio igual a 11.
- Existe una solución mejor, ya que se pueden introducir en la mochila los dos últimos objetos, obteniéndose un beneficio igual 12.

Minimización del tiempo en el sistema

Enunciado

Un único servidor (procesador, surtidor de gasolina, cajero de un banco) tiene que dar servicio a n clientes. Sea t_i el tiempo que requerirá el cliente i, con $1 \leq i \leq n$. Se desea minimizar el tiempo total T invertido en el sistema por todos los clientes:

$$T = \sum_{i=i}^{n} (\text{tiempo en el sistema para el cliente } i\ )$$

Como el número de clientes está fijado, minimizar el tiempo total en el sistema equivale a minimizar el tiempo medio invertido por cada cliente en el sistema.

Minimización del tiempo en el sistema

Ejemplo

- Supongamos que tenemos 3 clientes que requerirán los siguientes tiempos de servicio: $t_1 = 5$, $t_2 = 10$ y $t_3 = 3$.

- Existen 6 órdenes de servicio diferentes, siendo el tiempo total T invertido en el sistema por todos los clientes en cada uno de estos el siguiente:

$$
\begin{array}{lrcrcrcrcl}
1,2,3: & T & = & 5 & + & (5+10) & + & (5+10+3) & = & 38 \\
1,3,2: & T & = & 5 & + & (5+3) & + & (5+3+10) & = & 31 \\
2,1,3: & T & = & 10 & + & (10+5) & + & (10+5+3) & = & 43 \\
2,3,1: & T & = & 10 & + & (10+3) & + & (10+3+5) & = & 41 \\
3,1,2: & T & = & 3 & + & (3+5) & + & (3+5+10) & = & \mathbf{29} \\
3,2,1: & T & = & 3 & + & (3+10) & + & (3+10+5) & = & 34
\end{array}
$$

- La planificación óptima se obtiene cuando se sirve a los clientes por orden no decreciente de tiempo de servicio.

Minimización del tiempo en el sistema

Diseño del algoritmo voraz: elementos

- **Conjunto de candidatos:** los n clientes a los que hay que dar servicio.
- **Conjunto de candidatos elegidos:** la solución es un vector $(x_1, x_2, \cdots, x_n)$ que en cada x_i contiene al cliente al que se le dará servicio en i-ésimo lugar.
- **Función solución:** el número de clientes a los que se les ha determinado el orden en que se les dará servicio es igual a n.
- **Función de selección:** seleccionar uno de los clientes cuyo tiempo de servicio sea el menor de los restantes.
- **Función de factibilidad:** la selección de cualquiera de los restantes clientes da lugar a una solución factible.
- **Función objetivo:** tiempo total invertido en el sistema por todos los clientes:

$$\sum_{i=i}^{n} (\text{tiempo en el sistema para el cliente } i\ )$$

Diseño del algoritmo voraz: pseudocódigo

```
función planificar_clientes(t[1..n])
    // inicialización
    para todo i = 1, . . . , n hacer
        x[i] ← 0 // solución
    fin para todo
    // bucle voraz
    para todo i = 1, . . . , n hacer // función solución
        k ← cliente j con menor t[j] de los restantes // función de selección
        x[i] ← k
    fin para todo
    devolver x
fin función
```

- $T(n) \in O(n \log(n))$

Minimización del tiempo en el sistema

Teorema

Si se seleccionan los clientes por orden no decreciente de t_i, el algoritmo voraz encuentra la solución óptima.

Demostración (1/2)

- Sea $P = (p_1, p_2, \ldots, p_n)$ cualquier permutación de enteros del 1 al n, y sea $s_i = t_{p_i}$.

- Si se sirven clientes en el orden P, entonces el tiempo de servicio requerido por el i-ésimo cliente que haya que servir será s_i, y el tiempo total invertido en el sistema por todos los clientes es:

$$\begin{aligned} T(P) &= s_1 + (s_1 + s_2) + (s_1 + s_2 + s_3) + \ldots + (s_1 + s_2 + \ldots + s_n) \\ &= ns_1 + (n-1)s_2 + (n-2)s_3 + \ldots + 2s_{n-1} + s_n \\ &= \sum_{k=1}^{n} (n - k + 1)s_k \end{aligned}$$

Demostración (2/2)

- Supongamos ahora que P no organiza a los clientes por orden no decreciente de tiempos de servicio. Entonces, se pueden encontrar dos enteros a y b con $a < b$ y $s_a > s_b$. Si se intercambia la posición de estos dos clientes, se obtiene un nuevo orden de servicio Q que es simplemente el orden P después de intercambiar los enteros p_a y p_b.

- El tiempo total invertido en el sistema por todos los clientes si se emplea la planificación Q es:

$$T(Q) = (n - a + 1)s_b + (n - b + 1)s_a + \sum_{\substack{k=1 \\ k \neq a,b}}^{n} (n - k + 1)s_k$$

- Si se compara $T(P)$ con $T(Q)$, se concluye que la planificación nueva es preferible a la anterior:

$$\begin{aligned} T(P) - T(Q) &= (n - a + 1)(s_a - s_b) + (n - b + 1)(s_b - s_a) \\ &= (b - a)(s_a - s_b) > 0 \end{aligned}$$

Enunciado

Se tienen n trabajadores y n tareas. Sea c_{ij} el coste de asignar la tarea j al trabajador i. Una asignación de tareas puede expresarse como una asignación de los valores 0 y 1 a las variables x_{ij}, donde:

- $x_{ij} = 0$ significa que al trabajador i no le han asignado la tarea j.
- $x_{ij} = 1$ indica que sí se la han asignado.

Una asignación es válida si a cada trabajador solo le corresponde una tarea y cada tarea está asignada a un único trabajador.

Dada una asignación válida, se define su coste como:

$$\sum_{i=1}^{n} \sum_{j=1}^{n} x_{ij} c_{ij}$$

Una asignación es óptima si es de mínimo coste.

Diseño del algoritmo voraz: elementos

- **Conjunto de candidatos:** los n trabajadores (o las n tareas).
- **Conjunto de candidatos elegidos:** la solución es un vector $(x_1, x_2, \cdots, x_n)$ que en cada x_i contiene el trabajador (la tarea) que se asigna a la tarea (al trabajador) i.
- **Función solución:** se han asignado exactamente n trabajadores (tareas).
- **Función de factibilidad:** un mismo trabajador (tarea) solo se puede asignar a una única tarea (trabajador).
- **Función objetivo:** coste total de asignar cada trabajador (tarea) a cada tarea (trabajador):

$$\sum_{i=1}^{n} \sum_{j=1}^{n} x_{ij} c_{ij}$$

Asignación de tareas

Diseño del algoritmo voraz: función de selección

- **Estrategia 1.** Asignar a cada tarea el trabajador de menor coste de los restantes.
- **Estrategia 2.** Asignar a cada trabajador la tarea de menor coste de las restantes.

¿Es alguna de estas dos estrategias mejor que la otra?

¿Es alguna de estas dos estrategias óptima?

Estrategia de selección 1: ejemplo de aplicación

- Sean 4 tareas y 4 trabajadores. Los costes de la asignación entre cada tarea t_j y cada trabajador p_i son los siguientes:

	t_1	t_2	t_3	t_4
p_1	11	17	19	23
p_2	17	14	20	28
p_3	11	12	18	40
p_4	14	15	13	22

- El coste de la asignación es: 11 + 12 + 13 + 28 = **64**

Estrategia de selección 2: ejemplo de aplicación

- Sean 4 tareas y 4 trabajadores. Los costes de la asignación entre cada tarea t_j y cada trabajador p_i son los siguientes:

	t_1	t_2	t_3	t_4
p_1	11	17	19	23
p_2	17	14	20	28
p_3	11	12	18	40
p_4	14	15	13	22

- El coste de la asignación es: 11 + 14 + 18 + 22 = **65**

Óptimo

El algoritmo voraz no obtiene el óptimo para el problema de la asignación de tareas.

Demostración

- **Contraejemplo.** Algoritmo húngaro:

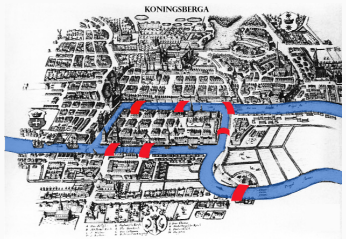
	t_1	t_2	t_3	t_4
p_1	11	17	19	23
p_2	17	14	20	28
p_3	11	12	18	40
p_4	14	15	13	22

- El coste de la asignación es: $11 + 14 + 13 + 23 = \mathbf{61}$

Teoría de grafos

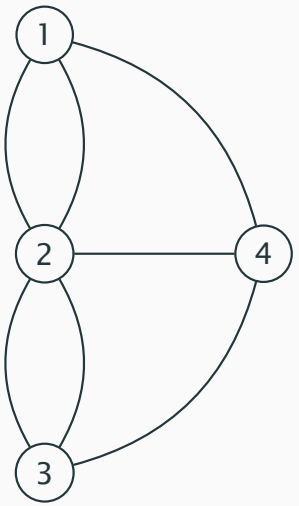
Teoría de grafos

- Surge de un trabajo del matemático suizo Leonhard Euler en 1736, basado en el problema de los puentes de Königsberg.
- Königsberg (Kaliningrado) era famosa por los siete puentes que unían ambos márgenes del río Pregel con dos de sus islas.
- El problema que se planteaba era si era posible, partiendo de un lugar arbitrario, regresar al lugar de partida cruzando cada puente una sola vez.

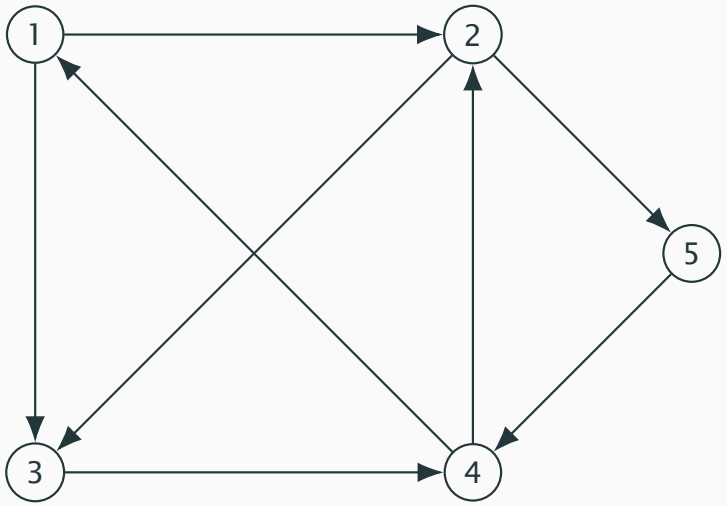


- Demostró que el grafo asociado al esquema de puentes de Königsberg no tiene solución.
- Resolvió un problema más general: las condiciones que debe satisfacer un grafo para garantizar que se puede regresar al vértice de partida sin pasar por la misma arista más de una vez.

- Un grafo simple G es un par (V, A) tal que:
  - V es un conjunto finito no vacío de vértices o nodos. Se llama orden de G al número de vértices de G y se denota n.
  - A es un subconjunto del producto cartesiano $V \times V$ (pares de vértices), cuyos elementos se denominan arcos (grafos dirigidos) o aristas (grafos no dirigidos), que conectan los vértices de V.
- En un grafo simple no existen auto ciclos ni multi-aristas.

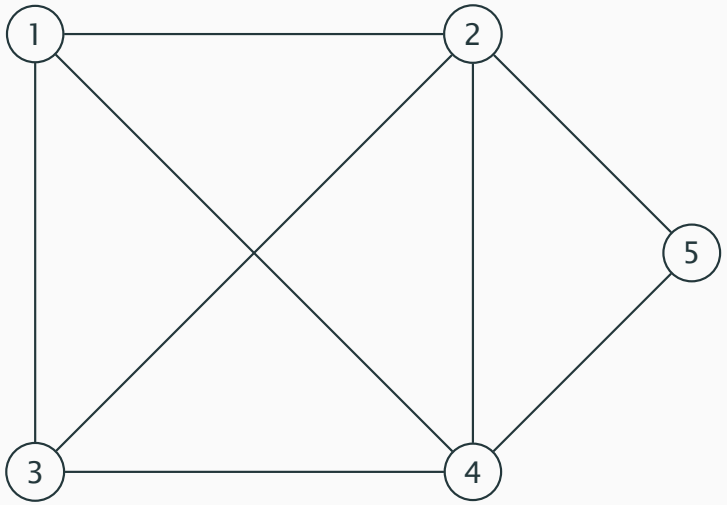


- En un grafo dirigido, cada arco viene dado por un par ordenado $\langle u, v \rangle$, donde $u \in V$ es el vértice inicial del arco y $v \in V$ es el vértice final.



- El máximo número de arcos de un grafo dirigido es $n(n-1)$. En tal caso, se dice que el grafo es completo.

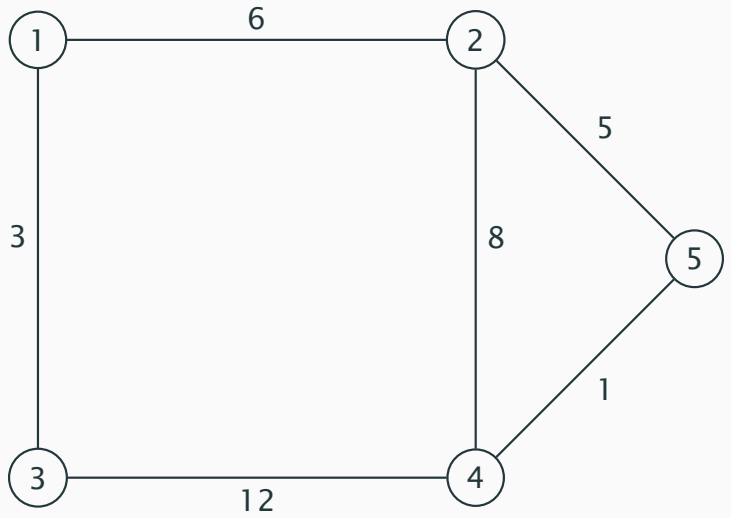
- En un grafo no dirigido, no se distinguen los vértices inicial y final. Dados dos vértices, $u, v \in V$, la arista (u, v) conecta dichos vértices en ambos sentidos.



- El máximo número de aristas de un grafo no dirigido es $n(n-1)/2$. En tal caso, se dice que el grafo es completo.

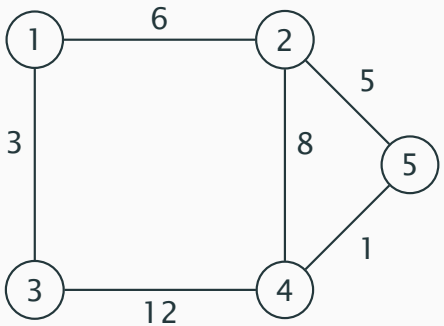
- El grado de un vértice es el número de arcos o aristas que inciden en dicho vértice.
- En grafos dirigidos, se definen el grado de entrada (número de arcos que entran al vértice) y el grado de salida (número de arcos que salen del vértice).
- Dos arcos o aristas se dice que son adyacentes si tienen un vértice en común.
- Dos vértices son adyacentes si existe algún arco o arista que los una.

- Los grafos ponderados son aquellos en los que los arcos o aristas llevan asignado un cierto valor real, denominado peso, que representa la distancia o el coste de desplazamiento entre dos vértices adyacentes.

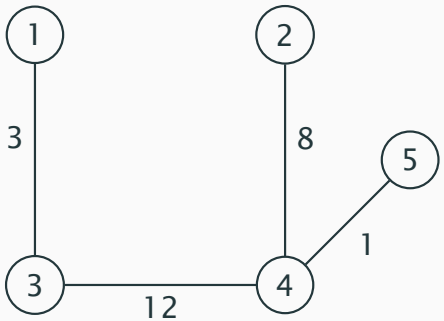


- Llamamos subgrafo de $G = (V, A)$ a un grafo $G_b = (V_b, A_b)$ tal que $V_b \subseteq V$ y $A_b = A \cap (V_b \times V_b)$. En el subgrafo G_b están todos los arcos o aristas que hay en A_b y que unen los vértices de V_b.

- Un camino entre los vértices u y v de un grafo no dirigido $G = (V, A)$ es una secuencia de vértices $u, w_1, \ldots, w_k, v$ tal que las aristas $(u, w_1), \ldots, (w_k, v) \in A$.
- En un grafo dirigido, un camino se define de la misma forma, pero usando arcos en lugar de aristas.
- Se denomina camino simple a un camino en el cual todos sus vértices son distintos salvo, quizás, el primero y el último.
- Se denomina ciclo a un camino simple en el cual los vértices primero y último son iguales.

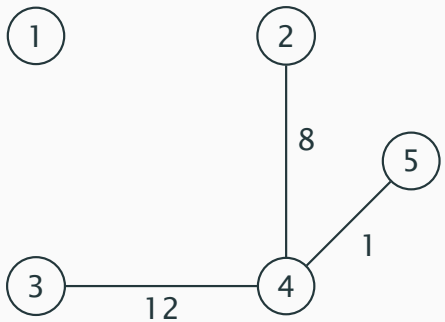


Grafo con ciclos

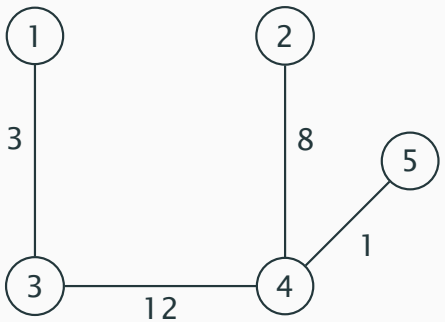


Grafo sin ciclos

- La longitud de un camino es el número de arcos o aristas que lo componen.
- En un grafo no dirigido, dos vértices u y v están conectados si existe al menos un camino entre ambos.
- Un grafo no dirigido es conexo si existe al menos un camino entre todo par de vértices.
- En un grafo dirigido, un vértice u es accesible desde otro vértice v si existe al menos un camino que parte de v y termina en u.
- Un grafo dirigido es fuertemente conexo si todo vértice u es accesible desde cualquier otro vértice v.



Grafo no conexo



Grafo conexo

Representación de grafos

- **Matriz de adyacencia.** Se usa una matriz M de tamaño $n \times n$ (siendo n el número de vértices del grafo) donde las filas y columnas hacen referencia a los vértices y cada elemento $M[i][j]$ almacena o bien un valor booleano que indica la conexión o no entre los vértices i y j, o bien el peso/coste de tal conexión (∞ si no existiera).

- **Lista de adyacencia.** Se usa una estructura L de tamaño n donde $L[i]$ contiene la lista de vértices adyacentes a i y, en el caso de grafos ponderados, también el peso/coste asociado a cada conexión.

Ejemplos de aplicación sobre grafos

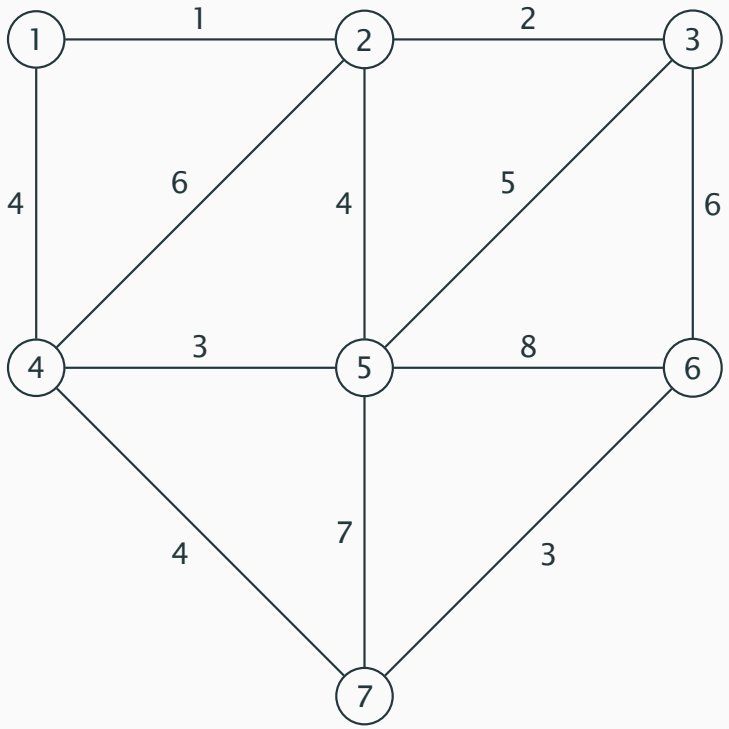
Árbol de recubrimiento de coste mínimo

Enunciado

Se considera un grafo no dirigido, ponderado y conexo $G = (V, A, c)$, donde $c : A \to \mathbb{R}^+$ es la función que asigna a cada arista (u, v) un coste $c(u, v) > 0$.

Se quiere encontrar el árbol de recubrimiento de G de coste mínimo, es decir, el subgrafo conexo acíclico (árbol) que contiene todos los vértices de G tal que la suma del coste de sus aristas sea mínima.

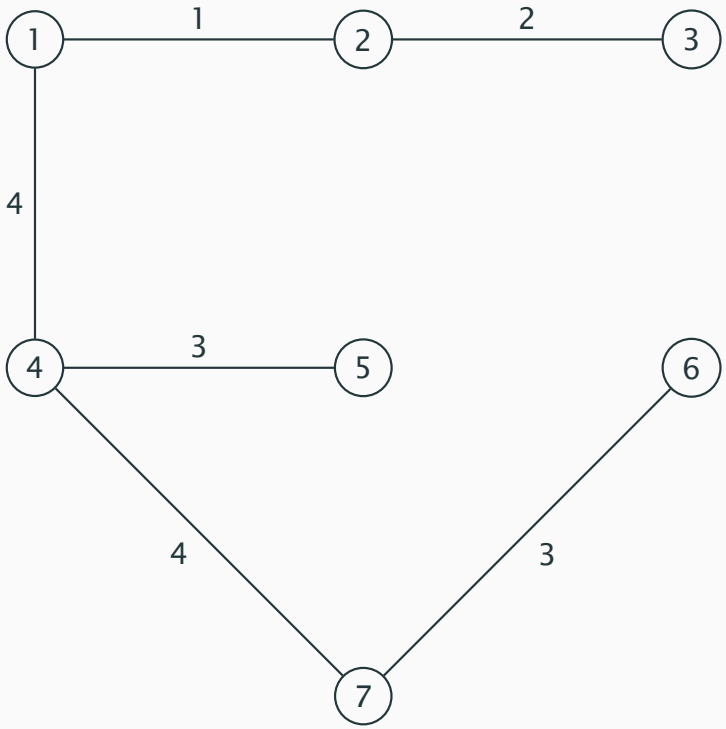
Árbol de recubrimiento de coste mínimo



Inicio

- Grafo no dirigido y conexo.
- Ponderado con pesos mayores que 0 (pesos positivos).

Árbol de recubrimiento de coste mínimo



Objetivo

- Un grafo parcial (subgrafo) acíclico y conexo.
- La suma de los pesos de las aristas debe ser la mínima posible.

Árbol de recubrimiento de coste mínimo

Estrategias de resolución

- **Algoritmo de Kruskal** (basado en aristas).
- **Algoritmo de Prim** (basado en vértices).

Algoritmo de Kruskal: idea general

- Inicialmente, la solución será un conjunto de aristas vacío.
- En cada paso, se selecciona la arista de menor coste de entre las aristas restantes que quedan en el conjunto de candidatos.
- Si la arista seleccionada no forma ciclos con las ya escogidas para formar parte de la solución, se añade a esta. En otro caso, se descarta y no se vuelve a considerar.
- El proceso anterior se repite hasta que la solución esté formada por $n - 1$ aristas.
- Como en cada paso se selecciona la arista de menor coste, conviene tenerlas ordenadas previamente.

Algoritmo de Kruskal: diseño voraz (elementos)

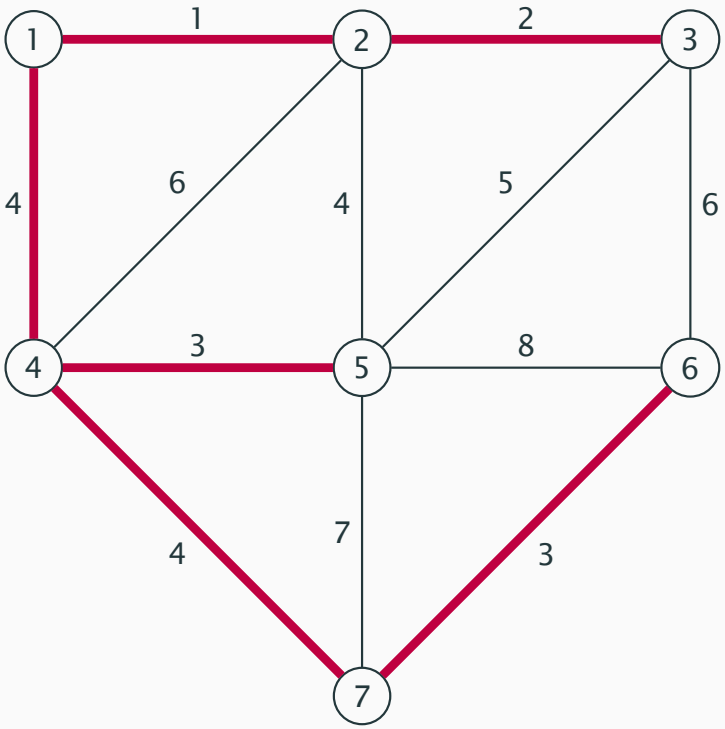
- **Conjunto de candidatos:** las aristas del grafo.
- **Conjuntos de candidatos elegidos:** la solución es un conjunto de aristas T elegidas del grafo original.
- **Función solución:** las aristas del conjunto solución T unen todos los vértices del grafo sin formar ciclos (árbol con $n - 1$ aristas).
- **Función de selección**: se selecciona la arista de menor coste de entre las restantes.
- **Función de factibilidad:** el conjunto de aristas elegidas no forma ciclos.
- **Función objetivo:** la suma de los pesos de las aristas que forman la solución.

Algoritmo de Kruskal: pseudocódigo

```
función kruskal(G = (V, A, c))
    // inicialización
    Ordenar A por orden creciente de coste
    n ←|V| // número de vértices de V
    T ← ∅ // contendrá las aristas de la solución
    Iniciar n conjuntos, cada uno con un elemento distinto de V
    mientras |T| < n−1 hacer // función solución
        // función de selección
        e ← arista (u,v) de menor coste de A aún no considerada
        // función de factibilidad
        compu ← buscar(u)
        compv ← buscar(v)
        si compu ≠ compv entonces
            fusionar(compu, compv)
            T ← T ∪ {e} // añadir la arista candidata a la solución
        fin si
    fin mientras
    devolver T
fin función
```

- $T(a, n) \in \Theta(a \log(n))$

Algoritmo de Kruskal: ejemplo de aplicación



Paso	Arista	¿Aceptada?	Componentes conexas
Inicialización	–	–	{1} {2} {3} {4} {5} {6} {7}
1	(1,2)	Sí	{1,2} {3} {4} {5} {6} {7}
2	(2,3)	Sí	{1,2,3} {4} {5} {6} {7}
3	(4,5)	Sí	{1,2,3} {4,5} {6} {7}
4	(6,7)	Sí	{1,2,3} {4,5} {6,7}
5	(1,4)	Sí	{1,2,3,4,5} {6,7}
6	(2,5)	No	{1,2,3,4,5} {6,7}
7	(4,7)	Sí	{1,2,3,4,5,6,7}

$T = \{(1,2), (1,4), (2,3), (4,5), (4,7), (6,7)\}$

Coste de la solución: **17**

Consideraciones

- Un conjunto de aristas factible es prometedor si se puede extender para producir no solo una solución, sino una solución óptima al problema del árbol de recubrimiento de coste mínimo.
- El conjunto vacío siempre es prometedor (siempre existe una solución óptima).
- Si un conjunto de aristas prometedor ya es una solución, la extensión requerida es irrelevante, y esta solución debe ser óptima.
- Una arista sale de un conjunto de vértices dado si tiene exactamente un extremo en el conjunto de vértices.

Lema

Sea $G = (V, A)$ un grafo no dirigido, ponderado y conexo. Sea $B \subset V$ un subconjunto estricto de los vértices de G. Sea $T \subseteq A$ un conjunto de aristas prometedor, tal que no haya ninguna arista de T que salga de B. Sea a la arista de menor coste que sale de B (o una de las de menor coste si hay empates). Entonces, $T \cup \{a\}$ es prometedor.

Demostración del lema

- Como T es prometedor, existe un árbol de recubrimiento de coste mínimo, U, de G, tal que $T \subseteq U$.
- Si $a \in U$, entonces no hay nada que probar.
- Si $a \notin U$, al añadir la arista a al árbol U se creará un ciclo. En este, puesto que a sale de B, existe necesariamente al menos otra arista d que también sale de B, o bien el ciclo no se cerraría.
- Si se elimina d, el ciclo desaparece y se obtiene un nuevo árbol W que recubre G.
- Sin embargo, el coste de a, por definición, no es mayor que el de d y, por tanto, el coste total de las aristas de W no sobrepasa el de las aristas de U. Por tanto, W es también un árbol de recubrimiento de coste mínimo de G y contiene a la arista a.
- La demostración se completa viendo que $T \subseteq W$, ya que la arista d que se ha eliminado sale de B y, por tanto, no podría haber sido una arista de T.

Teorema

El algoritmo de Kruskal obtiene un árbol de recubrimiento de coste mínimo.

Demostración (1/2)

- Por inducción sobre el número de aristas en T. Si T es prometedor, sigue siendo prometedor en cualquier fase del algoritmo cuando se le añade una arista adicional. Cuando el algoritmo termina, T da una solución al problema. Puesto que también es prometedora, la solución es óptima.

- **Base de la inducción.** El conjunto vacío es prometedor porque G es conexo y, por tanto, tiene que existir una solución.

Demostración (2/2)

- **Paso de inducción.** Supongamos que T es prometedor inmediatamente antes de que el algoritmo añada una nueva arista $a = (u, v)$. Las aristas de T dividen a los vértices de G en dos o más componentes conexas. El vértice u se encuentra en una de estas componentes y v está en otra. Sea B el conjunto de vértices de la componente que contiene a u:
  - El conjunto B es un subconjunto estricto de los vértices de G (por ejemplo, no incluye a v).
  - T es un conjunto prometedor tal que ninguna arista de T sale de B.
  - a es una de las aristas de coste mínimo que salen de B (todas las aristas de menor coste ya se han examinado y se han añadido a T o se han descartado porque tenían los dos extremos en la misma componente conexa).
- Entonces, se cumplen las condiciones del lema y se concluye que el conjunto $T \cup \{a\}$ también es prometedor.

Algoritmo de Prim: idea general

- Sea B un conjunto de vértices y sea T el conjunto de aristas que forman la solución.
- Inicialmente, B contiene un único vértice arbitrario, y T está vacío.
- En cada paso, se selecciona la arista (u, v) de menor coste tal que $u \in B$ y $v \in V \setminus B$.
- Entonces, se añade v a B y (u, v) a T (las aristas de T forman en todo momento un árbol de recubrimiento mínimo para los vértices de B).
- El proceso anterior continua mientras $B \neq V$.

Algoritmo de Prim: diseño voraz (elementos)

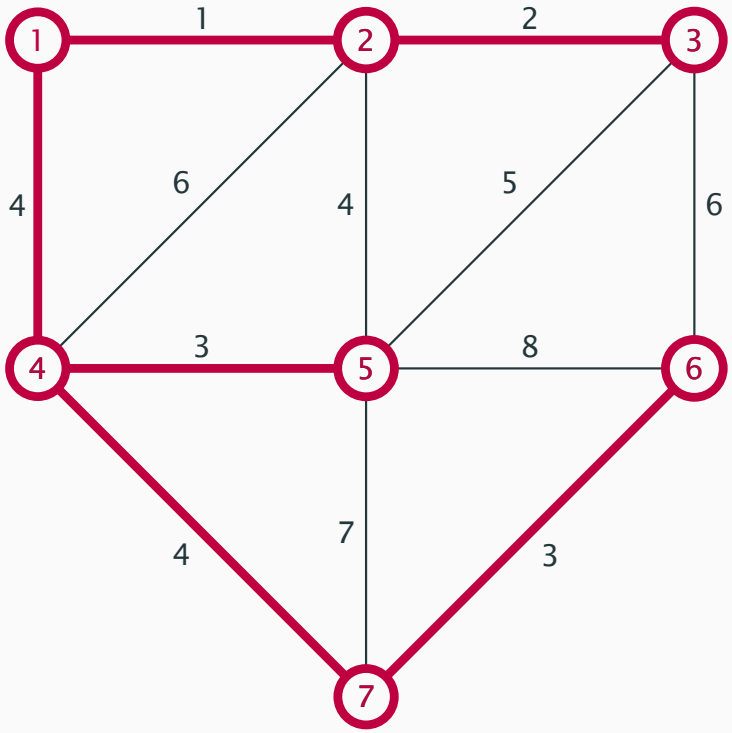
- **Conjunto de candidatos:** los vértices del grafo.
- **Conjuntos de candidatos elegidos:** los vértices que se han elegido para obtener las aristas que forman parte del conjunto solución T.
- **Función solución:** se han seleccionado n vértices y el conjunto de aristas que forman la solución conecta todos los vértices sin ciclos.
- **Función de selección**: se selecciona el vértice no utilizado aún tal que la arista que une este vértice con uno de los ya elegidos tiene coste mínimo.
- **Función de factibilidad:** no deben existir ciclos en las aristas que unen los vértices elegidos (siempre se cumple gracias a la función de selección).
- **Función objetivo:** la suma de los pesos de las aristas que forman la solución.

Algoritmo de Prim: pseudocódigo

```
función prim(G = (V, A, c))
    // inicialización
    T ← ∅ // contendrá las aristas de la solución
    B ← elemento arbitrario de V
    mientras |B| ≠ |V| hacer // función solución
        // función de selección
        buscar e = (u, v) de coste mínimo tal que u ∈ B y v ∈ V \ B
        T ← T ∪ {e} // añadir la arista candidata a la solución
        B ← B ∪ {v}
    fin mientras
    devolver T
fin función
```

- $T(n) \in \Theta(n^2)$

Algoritmo de Prim: ejemplo de aplicación



Paso	Arista	B
Inicialización	–	{1}
1	(1,2)	{1,2}
2	(2,3)	{1,2,3}
3	(1,4)	{1,2,3,4}
4	(4,5)	{1,2,3,4,5}
5	(4,7)	{1,2,3,4,5,7}
6	(7,6)	{1,2,3,4,5,6,7}

$T = \{(1,2), (1,4), (2,3), (4,5), (4,7), (6,7)\}$

Coste de la solución: **17**

Teorema

El algoritmo de Prim obtiene un árbol de recubrimiento de coste mínimo.

Demostración (1/2)

- Por inducción sobre el número de aristas en T. Si T es prometedor, sigue siendo prometedor en cualquier fase del algoritmo cuando se le añade una arista adicional. Cuando el algoritmo termina, T da una solución al problema. Puesto que también es prometedora, la solución es óptima.

- **Base de la inducción.** El conjunto vacío es prometedor.

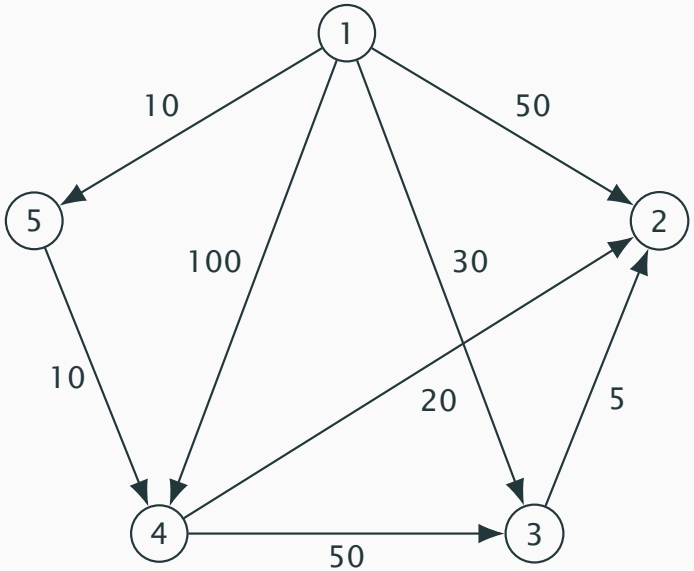
Demostración (2/2)

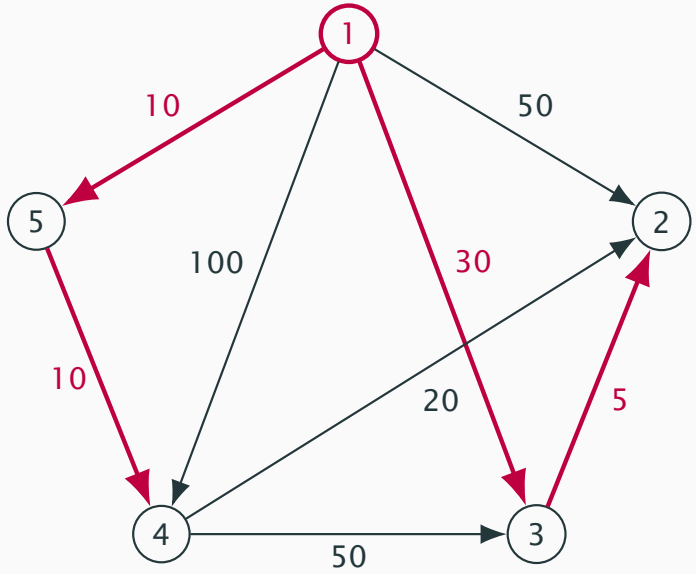
- **Paso de inducción.** Supongamos que T es prometedor inmediatamente antes de que el algoritmo añada una nueva arista $a = (u, v)$. Sea $\overline{B} = V \setminus B$ el conjunto de vértices de G que no están en B:
  - El conjunto $\overline{B}$ es un subconjunto estricto de V (ya que B lo es).
  - T es un conjunto prometedor tal que ninguna arista de T sale de $\overline{B}$.
  - a es por definición una de las aristas de coste mínimo que salen de $\overline{B}$ y conectan con B.
- Entonces, se cumplen las condiciones del lema y se concluye que el conjunto $T \cup \{a\}$ también es prometedor.

Enunciado

Sea $G = (V, A, c)$ un grafo dirigido y ponderado, donde $c : A \to \mathbb{R}^+$ es la función que asigna a cada arco $\langle u, v \rangle$ un coste $c(u, v) > 0$. Uno de sus vértices, que se denominará v_0, será el vértice origen. Se trata de encontrar el camino de coste mínimo que va desde v_0 a cualquier otro vértice v_i de G.

Caminos de coste mínimo





Objetivo

- Partir del vértice 1 (v_0).
- Encontrar el camino de coste mínimo que va desde vértice 1 al resto de vértices del grafo.

Algoritmo de Dijkstra: idea general (1/3)

- Se utilizan dos conjuntos de vértices, S y C.
- El conjunto S contiene los vértices que ya han sido elegidos (aquellos cuyo camino de coste mínimo desde el vértice v_0 ya es conocido):
  - Inicialmente, solo contiene a v_0.
  - Al finalizar, contiene todos los vértices del grafo.
- El conjunto C contiene todos los demás vértices, cuya distancia mínima desde el vértice origen todavía no se conoce, y que son candidatos a ser seleccionados en alguna etapa posterior.
- Por tanto, se cumple la propiedad de que $V = S \cup C$.
- En cada paso, se selecciona el vértice de C cuyo camino al origen sea de coste mínimo, y se añade a S.

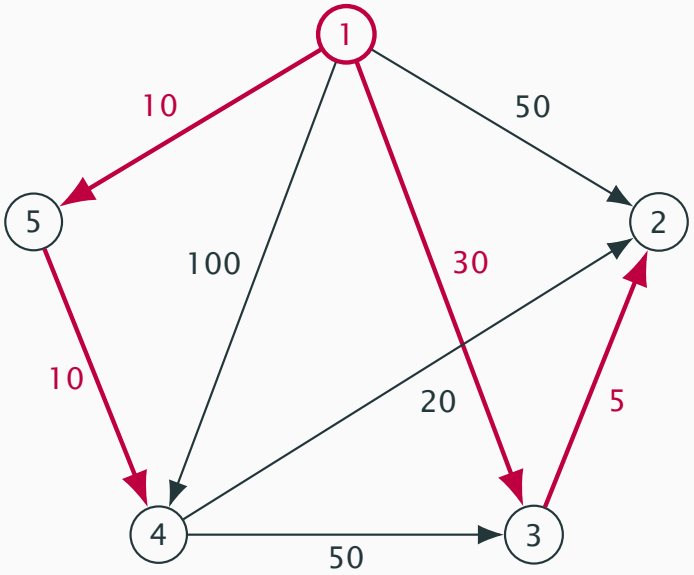
Algoritmo de Dijkstra: idea general (2/3)

- Un camino desde el vértice origen v_0 hasta otro vértice $v \in C$ es especial si todos los vértices intermedios del camino están en S.
- En cada etapa del algoritmo, existe un vector D que contiene el coste del camino especial de coste mínimo que va hasta cada vértice del grafo.
- Cuando se desea añadir un nuevo vértice v a S, el camino especial más corto hasta v es también el más corto de los caminos posibles hasta v. Inicialmente, $D[v_i] = c(v_0, v_i)$.
- Al finalizar el algoritmo, todos los vértices del grafo se encuentran en S (todos los caminos desde el vértice origen hasta algún otro vértice son especiales).
- Los valores que hay en D dan la solución del problema de caminos de coste mínimo.

Algoritmo de Dijkstra: idea general (3/3)

- Se supone que los vértices de G están numerados desde 1 hasta n (el vértice 1 es el v_0).
- La matriz L contiene el coste de todos los arcos:
  - $L[i][j] > 0$, si el arco $\langle i, j \rangle \in A$.
  - $L[i][j] = \infty$, en otro caso.
- Para determinar no solo el mínimo coste de los caminos sino también por dónde pasan, se añade el vector P, en donde $P[v_i]$ contiene el número del vértice que precede a v_i dentro del camino de mínimo coste. Inicialmente, $P[v_i] = v_0$.
- Para hallar el camino de mínimo coste, se siguen los punteros P hacia atrás, desde el destino hacia el origen.

Caminos de coste mínimo



Datos

- $D = [0, 35, 30, 20, 10]$.
- $P = [1, 3, 1, 5, 1]$.

¿Se puede reconstruir la ruta usando P?

Algoritmo de Dijkstra: diseño voraz (elementos)

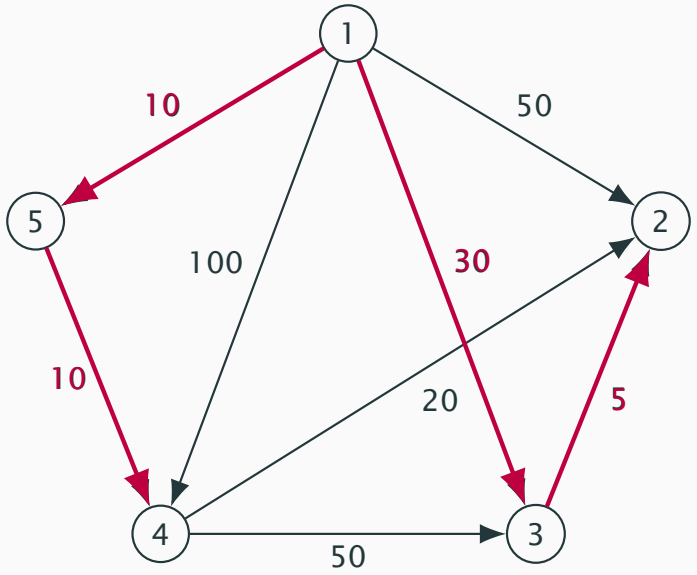
- **Conjunto de candidatos:** los vértices del grafo, excepto v_0.
- **Conjunto de candidatos elegidos:** vértices cuyo camino de coste mínimo desde v_0 ya se conoce (vértices añadidos a S).
- **Función solución:** se han elegido $n-1$ vértices del conjunto C para insertarlos en el conjunto S.
- **Función de selección**: se selecciona el vértice v_i no utilizado (no insertado en S) cuyo $D[v_i]$ sea mínimo.
- **Función de factibilidad:** siempre es factible insertar un vértice en la solución.
- **Función objetivo:** la suma de los pesos de las aristas que forman el camino del vértice v_0 a cada uno de los restantes vértices del grafo.

Algoritmo de Dijkstra: pseudocódigo

```
función dijkstra(L[1.. n][1.. n])
    // inicialización
    C ← 2, 3, . . . , n // S = N \ C solo existe implícitamente
    para todo i = 1, . . . , n hacer
        D[i] ← L[1][i]
        P[i] ← 1
    fin para todo
    // bucle voraz
    mientras C ≠ ∅ hacer // función solución
        // función de selección
        v ← algún elemento de C que minimiza D[v]
        C ← C \ {v} // implícitamente S ← S ∪ {v}
        para todo w ∈ C hacer
            si D[w] > D[v] + L[v][w] entonces
                D[w] ← D[v] + L[v][w]
                P[w] ← w
            fin si
        fin para todo
    fin mientras
    devolver D, P
fin función
```

- $T(n) \in \Theta(n^2)$

Caminos de coste mínimo



Paso	v	S	C	D	P
Inicio	-	$\{1\}$	$\{2,3,4,5\}$	$[0,50,30,100,10]$	$[1,1,1,1,1]$
1	5	$\{1,5\}$	$\{2,3,4\}$	$[0,50,30,20,10]$	$[1,1,1,5,1]$
2	4	$\{1,4,5\}$	$\{2,3\}$	$[0,40,30,20,10]$	$[1,4,1,5,1]$
3	3	$\{1,3,4,5\}$	$\{2\}$	$[0,35,30,20,10]$	$[1,3,1,5,1]$
3	2	$\{1,2,3,4,5\}$	$\{\emptyset\}$	$[0,35,30,20,10]$	$[1,3,1,5,1]$

Teorema

El algoritmo de Dijkstra halla los caminos de coste mínimo desde un vértice origen v_0 hasta los demás vértices del grafo.

Demostración (1/3)

- Se demostrará por inducción que:
  1. $v_i \in S$ $(v_i \neq v_0) \Rightarrow D[i]$ es el coste del camino óptimo desde v_0 a v_i.
  2. $v_i \notin S \Rightarrow D[i]$ es el coste del camino especial óptimo desde v_0 a v_i.

- **Base de la inducción.** Inicialmente, $S = \{v_0\}$. El único camino especial a cualquier vértice v_i es el directo, y D se inicia con esos costes.

- **Hipótesis de inducción.** Las propiedades (1) y (2) son válidas justo antes de añadir a S el vértice v que minimiza $D[v]$ entre los vértices $v \in C$.

Demostración (2/3)

- Paso de inducción para la situación (1):
  - Si $v_i \in S$ antes de añadir v a S, $v_i \in S$ también después de hacerlo, por lo que (1) seguirá siendo válida.
  - No obstante, antes de añadir v a S, es preciso comprobar que $D[v]$ es el coste del camino óptimo de v_0 a v.
  - La hipótesis de inducción dice que $D[v]$ es el coste del camino especial óptimo de v_0 a v. Por tanto, basta comprobar que el camino óptimo de v_0 a v no incluye ningún vértice que no pertenezca a S.
  - Sea u el primero de esos vértices. El camino que va desde v_0 a u es un camino especial y su coste, aplicando la hipótesis de inducción, será $D[u]$.
  - Claramente, el coste del camino que va de v_0 a v a través de u es mayor o igual que $D[u]$, es decir, $D[v] \geq D[u]$, ya que los costes de los arcos son positivos.
  - Por otra parte, $D[u] \geq D[v]$, ya que $v = arg\min_{\forall w \in C}(D[w])$.
  - Por tanto, el coste del camino que va de v_0 a v a través de u es, como mínimo, igual a $D[v]$ y no mejora el del camino especial óptimo de v_0 a v.

Demostración (3/3)

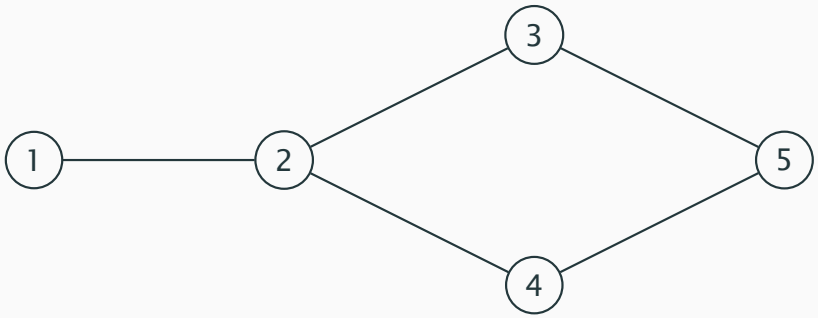
- Paso de inducción para la situación (2):
  - Considérese un vértice $u \notin S$, distinto de v. Cuando v se añade a S, hay dos opciones para el camino especial de coste mínimo desde v_0 hasta u: o bien no cambia, o bien ahora pasa a través de v (y, posiblemente, también a través de otros vértices de S).
  - En el segundo caso, sea x el último vértice de S visitado antes de llegar a u. El coste de ese camino es $D[x] + c(x, u)$.
  - A primera vista, podría parecer que para calcular el nuevo valor de $D[u]$ deberíamos comparar el valor actual de $D[u]$ con el de todos los posibles caminos a través de S: $D[x] + c(x, u)$ para todo $x \in S$. Sin embargo, esta comparación ya fue hecha en el momento de añadir x a S, para todo $x \neq v$, y $D[x]$ no ha variado desde entonces. Por tanto, basta comparar el valor actual de $D[u]$ con $D[v] + c(v, u)$ y quedarse con el mínimo.
  - Puesto que el algoritmo hace precisamente eso, la situación (2) seguirá siendo cierta después de añadir a S un nuevo vértice v.

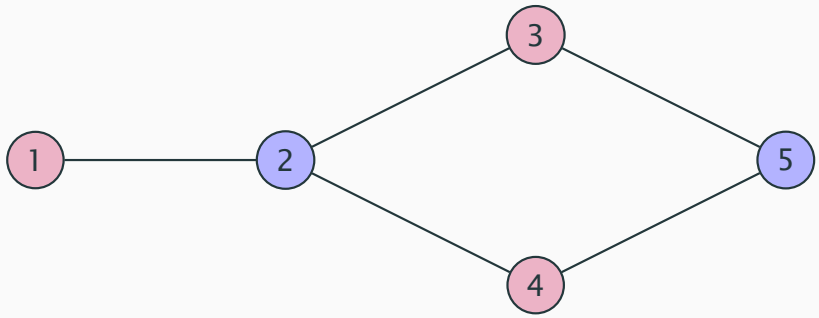
El coloreo de un grafo

Enunciado

Sea $G = (V, A)$ un grafo no dirigido. Se desea asignar un color a cada vértice del grafo de forma que:

- No haya dos vértices adyacentes con el mismo color.
- Se utilice el mínimo número de colores posibles.





Teorema de los cuatro colores (Appel y Haken, 1976)

Un grafo plano requiere a lo sumo cuatro colores para pintar sus vértices de modo que no haya vértices adyacentes con el mismo color.

Complejidad

- Clase de problemas P. Aquellos que son resolubles en un tiempo igual o inferior a orden polinómico $\mathrm{O}(n^k)$.

- Clase de problemas NP. Aquellos cuya solución es fácil de verificar, pero cuyo cálculo conlleva una complejidad muy grande debido a la gran cantidad de potenciales soluciones (óptimas y no óptimas) a explorar.

- El problema del coloreo de un grafo es NP, por lo que no se conoce una solución eficiente para resolverlo.

Heurísticas

- Al no conocerse soluciones óptimas eficientes a los problemas NP, si la eficiencia es un requisito en la implementación de la solución, se aceptan soluciones «eficientemente buenas» o «las mejores que se pueden encontrar».

- Las heurísticas son criterios, métodos, o principios para decidir cuál, entre una serie de alternativas de acción, promete ser más efectiva para lograr una meta concreta.

- Las heurísticas representan un compromiso entre dos exigencias:
  - Desde el punto de vista computacional, deben ser simples.
  - Deben discriminar correctamente entre buenas y malas opciones.

El coloreo de un grafo

Idea general

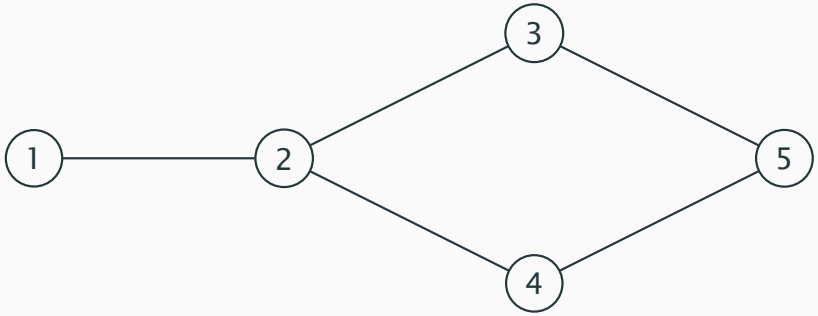
- Inicialmente, suponemos que ningún vértice del grafo está coloreado.
- Se selecciona, al azar, un vértice del grafo y se pinta de un color.
- Se escogen todos los vértices no adyacentes del vértice seleccionado que aún no estén coloreados y se pintan del mismo color.
- Se repite el procedimiento anterior hasta que todos los vértices del grafo estén pintados.

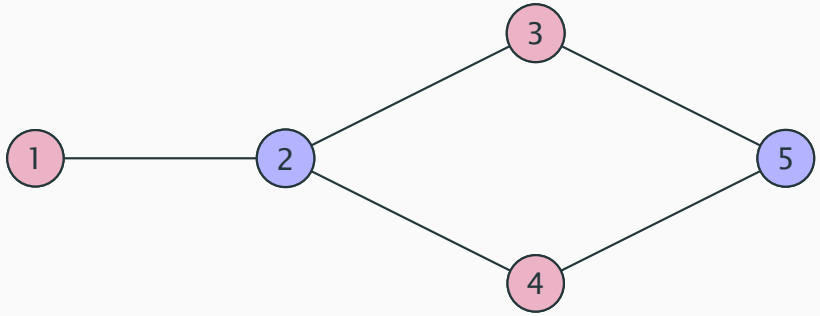
El coloreo de un grafo

Diseño del algoritmo voraz: elementos

- **Conjunto de candidatos:** los n vértices del grafo.
- **Conjuntos de candidatos elegidos:** los vértices ya coloreados.
- **Función solución:** los n vértices del grafo están coloreados.
- **Función de selección:** se selecciona un vértice al azar de los aún no coloreados.
- **Función de factibilidad:** no puede haber dos vértices adyacentes pintados del mismo color.
- **Función objetivo:** el número de colores usados para colorear los vértices del grafo.

El coloreo de un grafo





Pasos del algoritmo

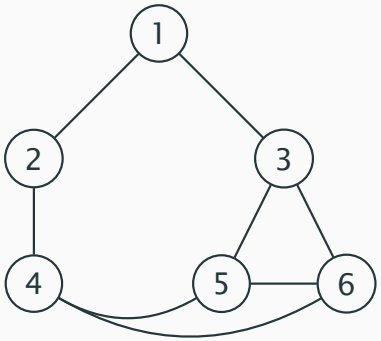
- Seleccionar el vértice 3 (al azar) y colorearlo.
- Rellenar también 1 y 4 con el mismo color.
- Seleccionar el vértice 2 (al azar) y colorearlo.
- Rellenar también 5 con el mismo color.

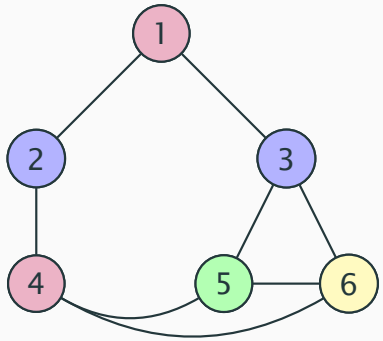
Óptimo

El algoritmo voraz para el problema del coloreo de un grafo no obtiene el óptimo.

Demostración (1/2)

- **Contraejemplo:**

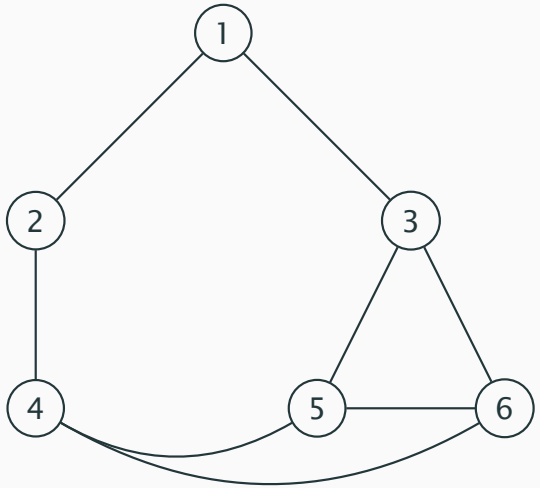


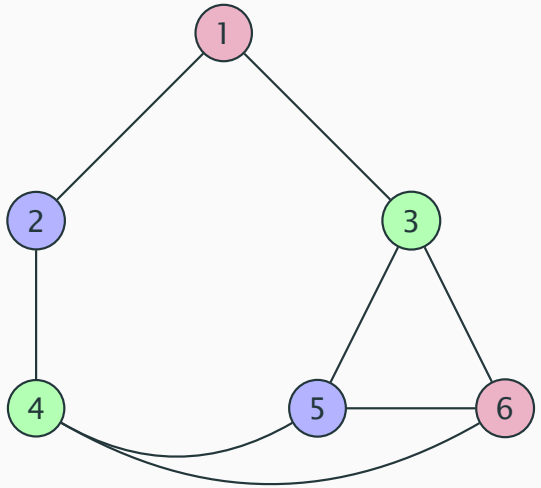


- Seleccionar el vértice 1 (al azar) y colorearlo. Rellenar también 4 con el mismo color. Seleccionar el vértice 2 (al azar) y colorearlo. Rellenar también 3 con el mismo color. Seleccionar el vértice 5 (al azar) y colorearlo. Seleccionar el vértice 6 (al azar) y colorearlo.

Demostración (2/2)

- **Contraejemplo:**





Se puede colorear con **tres colores** (solución óptima).

El viajante de comercio

Enunciado

Se conocen las distancias entre un determinado número de ciudades. Partiendo de una de ellas, un viajante debe visitar cada ciudad exactamente una vez y regresar al punto de partida habiendo recorrido en total la menor distancia posible.

Enunciado formal

Dado un grafo $G = (V, A)$ no dirigido, conexo, ponderado y completo, y dado uno de sus vértices v_0, encontrar el ciclo hamiltoniano de coste mínimo que comienza y termina en v_0.

El viajante de comercio

Complejidad

- El problema del viajante de comercio es NP, por lo que no se conoce una solución eficiente para resolverlo.

Heurísticas

- Vértices como candidatos. Escoger el vértice más cercano al último vértice añadido, siempre que no se haya seleccionado previamente y que no cierre el camino antes de pasar por todos los vértices.

- Aristas como candidatos. Como en el algoritmo de Kruskal, pero garantizando que al final se forme un ciclo y que de cada vértice no salgan más de dos aristas.

El viajante de comercio

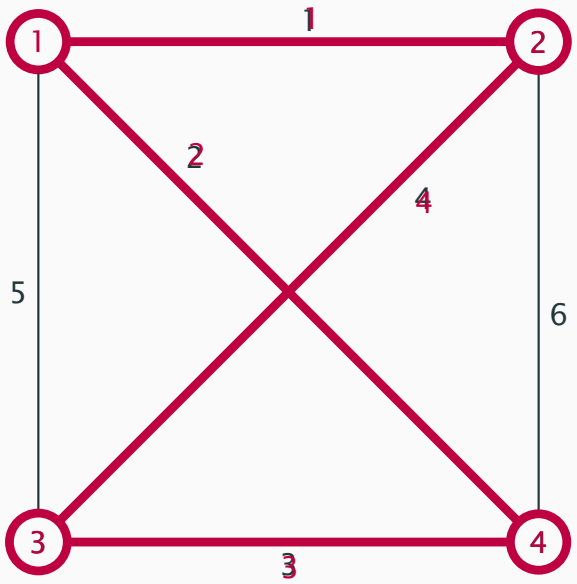
Idea general

- Inicialmente, suponemos que se parte de un vértice cualquiera del grafo.
- A continuación, se selecciona el vértice adyacente no visitado más cercano, e insertamos la arista que lo une en la solución.
- Al finalizar, se une la arista del último vértice seleccionado con el primero para cerrar el ciclo.

Diseño del algoritmo voraz: elementos

- **Conjunto de candidatos:** los vértices del grafo.
- **Conjuntos de candidatos elegidos:** los vértices ya elegidos.
- **Función solución:** se han elegido todos los vértices del grafo.
- **Función de selección:** se selecciona un vértice tal que la arista que lo une con el último vértice elegido tiene coste mínimo.
- **Función de factibilidad:** no se pueden formar ciclos (excepto al añadir el último vértice).
- **Función objetivo:** el coste del ciclo hamiltoniano.

El viajante de comercio



Paso	v	S	C	Arista	T
Inicio	-	1	$\{2,3,4\}$	-	-
1	2	1	$\{3,4\}$	(1,2)	(1,2)
2	3	2	$\{4\}$	(2,3)	(1,2), (2,3)
3	4	3	$\{\emptyset\}$	(3,4)	(1,2), (2,3), (3,4)
4	1	4	$\{\emptyset\}$	(4,1)	(1,2), (2,3), (3,4), (4,1)

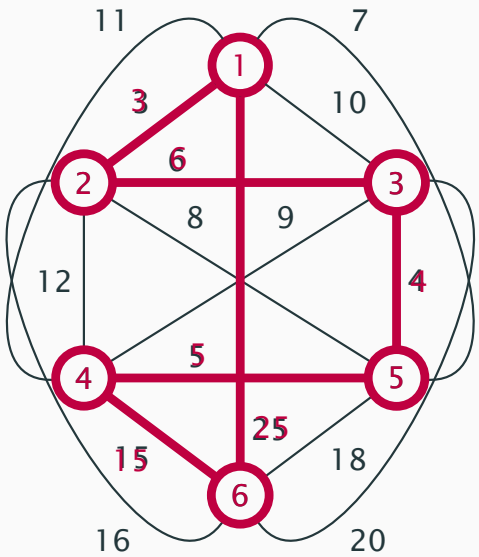
Coste: 1 + 4 + 3 + 2 = **10** (solución óptima)

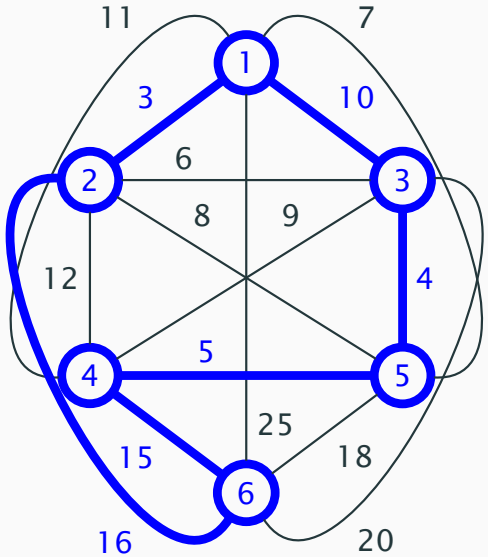
Óptimo

El algoritmo voraz para el problema del viajante de comercio no obtiene el óptimo.

Demostración

- **Contraejemplo:**





- Coste de la solución del algoritmo voraz: 3 + 6 + 4 + 5 + 15 + 25 = **58**
- Coste de la solución óptima: 3 + 16 + 15 + 5 + 4 + 10 = **53**

Consideraciones finales

Consideraciones finales

¿Por qué no emplear siempre algoritmos voraces?

- No todos los problemas admiten esta estrategia de resolución.
- La búsqueda de óptimos locales no tiene por qué conducir a un óptimo global.

Estrategia voraz: limitaciones

- Trata de ganar todas las batallas sin ser consciente de que para ganar la guerra muchas veces se necesita perder alguna batalla.
- Lo que parece bueno hoy no siempre tiene por qué ser bueno en el futuro.

Consideraciones finales

- Es fundamental encontrar la función de selección adecuada que garantice que el candidato escogido o rechazado es el que debe formar parte o no de la solución óptima sin posibilidad de reconsiderar tal decisión.

- Al plantear un algoritmo voraz (que por diseño será eficiente), hay que demostrar formalmente que conduce a la solución óptima del problema en todos los casos.

- En el caso de que no conduzca a soluciones óptimas, hay que presentar un contraejemplo para el cual el algoritmo no proporcione la solución óptima.

Consideraciones finales

- Debido a su eficiencia, los algoritmos voraces se usan incluso cuando no encuentran la solución óptima.
- A veces, es imprescindible encontrar pronto, o en un tiempo finito, una solución razonablemente buena, aunque no sea la óptima.
- Otras veces, un algoritmo voraz da una solución rápida (no óptima) del problema y, a partir de ella, aplicando algoritmos más sofisticados, la solución óptima se obtiene más rápidamente.
- Los algoritmos voraces se aplican en muchas circunstancias debido a su eficiencia, aunque solo den una solución aproximada al problema de optimización planteado.

Bibliografía

Bibliografía

Aclaración

- El contenido de las diapositivas es esquemático y representa un apoyo para las clases teóricas.

- Se recomienda completar los contenidos del tema 3 con apuntes propios tomados en clase y con la bibliografía principal de la asignatura.

Por ejemplo


G. Brassard and P. Bratley.
***Fundamentals of Algorithmics*.**
Prentice Hall, Englewwod Cliffs, New Jersey, 1996.


T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein.
***Introduction to Algorithms*.**
The MIT Press, Cambridge, MA, 2022.


J. L. Verdegay.
***Lecciones de Algorítmica*.**
Editorial Técnica AVICAM, 2017.